

PROJECT REPORT

VESTIGE AI

Memory That Doesn't Resign

Centralized CRM & AI-Powered Organizational Memory for Software Houses and Agencies

Prepared by

Zain Ul Aabideen

AI Engineer Intern, Dynamic Leo

Student, University of Engineering and Technology (UET) Taxila

3 August 2026

Contents

- Contents 2
- INTRODUCTION 3
- 1. Product Profile 4
 - 1.1 Problem and Product 4
 - 1.2 Ideal Customer Profile 4
 - 1.3 User Personas 4
 - 1.4 Jobs-To-Be-Done 5
 - 1.5 Business Rules 5
 - 1.6 Project Constraints 6
 - 1.7 Feature Scope 7
 - 1.8 Monetization and Success Metrics 7
- 2. Software Requirements Specification 9
 - 2.1 System Architecture 9
 - 2.2 Functional Requirements 9
 - 2.3 Database Schema (SQL Server) 11
 - 2.4 Vector and Graph Model 12
 - 2.5 API Structure 12
 - 2.6 Non-Functional Requirements 13
- 3. PX Strategy 15
 - 3.1 Screen Inventory 15
 - 3.2 Key Screen Layouts 15
 - 3.3 State Management 16
 - 3.4 API-to-UI Data Binding 16
 - 3.5 Access Control in the Interface 17
- 4. Context Handoffs 18
 - 4.1 What Is Bundled 18
 - 4.2 Handoff Rules 18
 - 4.3 Build Sequence 18
 - 4.4 Risks 19

INTRODUCTION

Vestige AI is a centralized CRM and AI-powered organizational memory platform for software houses and agencies. When an employee leaves mid-project, the client and project context carry leaves with them. Vestige AI captures that context from sources teams already use — WhatsApp, Telegram and email exports, and Google Meet recordings — and makes it searchable through a CRM dashboard and a RAG chat assistant that answers with citations.

This report is organized as four documents: the Product Profile (business), the SRS (technical), the PX Strategy (interface), and the Context Handoffs (how the three feed the build).

1. Product Profile

The master business document: business rules, personas, Jobs-To-Be-Done and project constraints.

1.1 Problem and Product

- Client and project context live in employees' chats, inboxes and memory — not in any structured system.
- When an employee exits, client history, technical decisions and commercial context are lost at once.
- Notion, ClickUp, Confluence and Jira require manual documentation discipline that breaks under delivery pressure. Jira tracks tasks, not relationships or reasoning.
- Vestige AI ingests raw informal communication, connects it into a knowledge graph, and makes it answerable in plain language — purpose-built to keep client continuity independent of any single employee.

1.2 Ideal Customer Profile

Dimension	Profile
Firm size and type	10–150 employees; software and digital agencies, IT outsourcing firms with multiple concurrent client projects.
Pain points	High turnover; communication split across WhatsApp, Telegram and email; founder is the primary source of institutional knowledge.
Buying triggers	A bad handover; scaling past founder capacity; a client complaint about lost context.
Buying roles	Owner/Founder (economic buyer), Delivery/Ops Manager (champion), PMs, account managers and engineers (end users).

1.3 User Personas

Persona	Needs From Vestige AI
Agency Owner / Founder	To stop being the single point of failure; pipeline, payments and delivery risk on one screen.
Delivery / Ops Manager	To reassign a project instantly without a week of handover meetings.
Project / Account Manager	One place to upload chats and meetings, and to retrieve what was promised to whom.
Engineer	Low-friction logging of completed work and technical decisions.
Replacement Joiner	Plain-language answers with citations, within hours instead of weeks.

1.4 Jobs-To-Be-Done

ID	Job-To-Be-Done
JTBD-01	When an employee resigns from the mid-project, keeps the project's full context in the system so work can be reassigned without losing days.
JTBD-02	When taking over a client, ask what has been agreed and what remains, to avoid re-asking answered questions.
JTBD-03	Attach WhatsApp and email history to the right client and project so it is findable later.
JTBD-04	Transcribe and summarize meetings against the project so decisions are recoverable without rewatching.
JTBD-05	Show the owner pipeline, payments and workload on one screen.
JTBD-06	Capture engineers' technical decisions as they happen so the reasoning survives months later.

1.5 Business Rules

ID	Rule
BR-01	Every ingested artefact attaches to exactly one client and, where applicable, one project.

ID	Rule
BR-02	Every AI answer displays source citations linking back to the original chat, transcript or log.
BR-03	AI output is advisory — a starting point for a human, never a final authority.
BR-04	Capture must need minimal deliberate effort; features depending on documentation discipline are out of scope.
BR-05	Payment and commercial data is role-restricted and never surfaced to users without permission.
BR-06	Data enters only by deliberate manual upload in Phase 1; no live platform API sync.
BR-07	A project stays fully readable and answerable after its assigned employee is removed.
BR-08	Retrieval is filtered by the requester's permissions before generation, not after.
BR-09	Positioned to the team as protection for their work, never as surveillance.
BR-10	Leads, clients, projects, employees and payments form one connected record; no orphan copies.

1.6 Project Constraints

ID	Constraint	Consequence
CON-01	Single developer, full time	Strictly sequential build; no parallel workstreams.
CON-02	Six weeks to a usable MVP	Scope frozen at the feature list in 1.7.
CON-03	Open-source, self-hostable components only	No paid managed AI services.
CON-04	Manual ingestion only in Phase 1	No OAuth, no API integration, no sync workers.
CON-05	Sample and dummy data for testing	Accuracy validated by spot-check, not production evaluation.
CON-06	Modest compute for inference	Small open-weight models, batch and async processing.

1.7 Feature Scope

Feature	What It Does
Lead Management	Track leads through pipeline stages until conversion.
Client Management	Central profile: contacts, active projects, relationship history.
Payment Tracking	Invoices, milestones, amounts paid and due per client and project.
Communication Log	Upload WhatsApp, Telegram and email exports, attached to the right client and project.
Meeting Transcripts	Upload Meet recordings; transcribe and summarise.
Project Management	Scope, milestones and status per project.
Employee Management	Roles, skills, and current and past project history.
Project Assignment	Assign and reassign employees instantly.
Work Logging	Employees log completed work and technical decisions.
Semantic Search	Search by meaning across chats, transcripts and logs.
AI Chat Assistant (RAG)	Plain-language answers from the team's own data, with source references.
Role-Based Access Control	Control who sees what, especially payments and sensitive client data.
Dashboard	Pipeline, active projects, payments due and workload on one screen.

Out of scope for the MVP: live WhatsApp/Gmail/Meet API sync, multi-tenant SaaS isolation and billing, mobile apps, model fine-tuning, automated RAG evaluation harnesses.

1.8 Monetization and Success Metrics

Phase	Model
Phase 1	Internal use — no pricing; tracked as internal ROI (time saved).

Phase	Model
Phase 2	Tiered per-seat — basic CRM tier versus Pro/AI tier (search plus chatbot) for pilot agencies.
Phase 3	Multi-tenant SaaS — per-seat subscription plus usage-based add-on for heavy AI compute.

Success metrics:

- Time-to-onboard a replacement employee onto an existing client project — the primary measure.
- Client complaints or escalations tied to handover and communication gaps.
- Upload completeness: communications and meetings ingested versus total that occurred.
- Weekly chatbot usage, and spot-checked answer accuracy with valid citations.
- Post-SaaS: pilot-to-paid conversion, MRR, churn and gross margin.

2. Software Requirements Specification

The technical blueprint: developer-facing rules, database schema and API structure, defined before any interface work.

2.1 System Architecture

Layer	Responsibility	Components
Ingestion	Accept raw material and bind it to a client and project.	Upload interface for chat, email and Meet files; work-log entry; format parsers.
AI Processing	Convert raw material into structured, embedded, summarized knowledge.	Whisper (transcription); chunking and BGE embeddings; Llama/Mistral (summarization, entity extraction).
Memory Storage	Persist structured, semantic and relational views.	SQL Server (CRM data); Qdrant (vectors); Neo4j (knowledge graph).
Delivery	Serve knowledge back, permission filtered.	React frontend; RAG chatbot combining vector and graph retrieval, answering with citations.

Component	Technology
Backend API	FastAPI (Python)
Structured data	SQL Server
Frontend	React
Vector search	Qdrant
Knowledge graph	Neo4j Community
Transcription	Whisper (Hugging Face)
Embeddings	BGE models (Hugging Face)
RAG generation	Llama / Mistral (open-weight)

2.2 Functional Requirements

ID	The system shall...
FR-CRM-01	Allow creation, editing and stage-tracking leads until conversion to a client.
FR-CRM-02	Maintain one client profile containing contacts, active projects and relationship history.
FR-CRM-03	Allow creation of projects linked to exactly one client, with scope, milestones and status.
FR-CRM-04	Maintain employee profiles containing role, skills and project history.
FR-CRM-05	Allow assignment and reassignment of employees without altering project history (BR-07).
FR-CRM-06	Record invoices and milestone payments per client and project, with paid and outstanding amounts.
FR-CRM-07	Present a dashboard summarizing pipeline, active projects, payments due and workload.
FR-ING-01	Accept manual upload of WhatsApp, Telegram and email export files.
FR-ING-02	Accept manual upload of Google Meet recordings and transcript exports.
FR-ING-03	Require every upload to be tagged to a client, and to a project where one applies (BR-01).
FR-ING-04	Parse each supported export into normalized message records with sender, timestamp and text.
FR-ING-05	Allow employees to log completed work and technical decisions against a project.
FR-ING-06	Quarantine unrecognized formats with a clear message rather than failing silently.
FR-AI-01	Transcribe uploaded meeting audio with Whisper and stored the transcript against the project.
FR-AI-02	Chunk all text content and generate embeddings using a BGE model.
FR-AI-03	Store embeddings in Qdrant with metadata for client, project, source type and date.
FR-AI-04	Generate a summary for each chat, transcript and work log, attached to the parent project.

ID	The system shall...
FR-AI-05	Extract entities and decisions — client, deadline, scope change, technology, commitment.
FR-AI-06	Write extracted entities and relationships into the Neo4j knowledge graph.
FR-AI-07	Process ingestion jobs asynchronously and expose job status to the uploading user.
FR-RAG-01	Provide semantic search by meaning across all chats, transcripts and work logs.
FR-RAG-02	Provide a chat interface accepting plain-language questions.
FR-RAG-03	Answer using Qdrant retrieval combined with related entities from Neo4j.
FR-RAG-04	Attach a citation list to every answer, linking to the source artefact (BR-02).
FR-RAG-05	Apply the requester's permission filter before retrieval, not after generation (BR-08).
FR-RAG-06	State clearly when no supporting source was found rather than generating an unsupported answer.
FR-SEC-01	Authenticate every user and issue a scoped session token.
FR-SEC-02	Enforce role-based access control on every API endpoint, not only in the interface.
FR-SEC-03	Restrict payment and commercial data to the owner and finance roles (BR-05).
FR-SEC-04	Record an audit entry for every data access, upload and permission change.

2.3 Database Schema (SQL Server)

Table	Key Fields
users	id, name, email, password_hash, role_id, is_active
roles	id, name, permission_set
employees	id, user_id, designation, skills, joined_on, status
leads	id, company, contact, source, stage, owner_id, created_at

Table	Key Fields
clients	id, name, industry, primary_contact, converted_from_lead_id
projects	id, client_id, title, scope, status, start_date, target_date
project_assignments	id, project_id, employee_id, role_on_project, from_date, to_date
work_logs	id, project_id, employee_id, summary, technical_notes, logged_at
payments	id, client_id, project_id, invoice_no, amount, due_date, paid_on, status
uploads	id, client_id, project_id, source_type, file_name, uploaded_by, status
messages	id, upload_id, sender, sent_at, content, chunk_ref
transcripts	id, upload_id, project_id, text, duration, generated_summary
audit_log	id, user_id, action, entity, entity_id, occurred_at

2.4 Vector and Graph Model

- **Qdrant** — one collection of chunk embeddings; payload carries client_id, project_id, source_type, source_id, sender, timestamp and visibility. Permission filters are applied as payload filters at query time (FR-RAG-05).
- **Neo4j nodes** — Client, Project, Employee, Meeting, Decision, Commitment, Technology, Deadline.
- **Neo4j relationships** — OWNS, ASSIGNED_TO, DISCUSSED_IN, DECIDED_IN, COMMITTED_TO, USES, DUE_ON, REPLACED_BY.

2.5 API Structure

Method and Path	Purpose
POST /api/v1/auth/login	Authenticate and return a scoped token.
GET, POST, PATCH /api/v1/leads	List, create and advance leads through pipeline stages.
GET, POST /api/v1/clients	List and create clients; /{id} returns the full client view.
GET, POST /api/v1/projects	List and create projects; /{id} returns scope, milestones and status.

Method and Path	Purpose
POST /api/v1/projects/{id}/assignments	Assign or reassign an employee.
GET, POST /api/v1/employees	List and maintain employee profiles and skills.
GET, POST /api/v1/payments	Record and list invoices and milestone payments (restricted role).
POST /api/v1/uploads	Upload a chat, email or meeting file with client and project tags.
GET /api/v1/uploads/{id}/status	Poll asynchronous processing status.
POST /api/v1/work-logs	Create a work log entry against a project.
GET /api/v1/search	Semantic search across ingested content, permission-filtered.
POST /api/v1/chat	Submit a question; returns an answer with citations.
GET /api/v1/dashboard	Aggregated pipeline, project, payment and workload counters.

2.6 Non-Functional Requirements

Category	Requirement
Security	Encryption in transit and at rest; password hashing; scoped tokens; audit logging of all access.
Access control	Role permissions enforced at the API layer and mirrored in the interface; retrieval filtered pre-generation.
Performance	CRM screens respond within roughly two seconds; AI ingestion runs asynchronously and never blocks the interface.
Reliability	Failed ingestion jobs are retryable and leave no partial embeddings in Qdrant.
Traceability	Every AI-generated statement resolves to at least one stored source record.

Category	Requirement
Portability	All components self-hostable; no dependency on a paid managed AI service.
Usability	Uploading and tagging a file takes no more than three interactions (BR-04).

3. PX Strategy

The bridge from backend logic to interface: screens, layouts, state management and API-to-UI data binding.

3.1 Screen Inventory

Screen	Purpose	Roles
Login	Authenticate and establish role scope.	All
Dashboard	Pipeline, active projects, payments due, workload.	Owner, Manager
Leads	Pipeline board by stage with conversion action.	Owner, Manager, PM
Client 360	Contacts, projects, communications, payments, history.	Owner, Manager, PM
Project Workspace	Scope, milestones, team, work logs, ingested material.	All
Upload Centre	Upload files with client and project tagging.	Manager, PM
Work Log Entry	Fast entry of completed work and decisions.	Engineer, PM
Employees	Profiles, skills, current and past assignments.	Owner, Manager
Payments	Invoices, milestones, amounts due and paid.	Owner, Finance
Search	Semantic results with source and project context.	All
AI Assistant	Conversational answers with visible citations.	All

3.2 Key Screen Layouts

- **Dashboard** — four counter cards across the top (open leads, active projects, payments due, unassigned work); project status list on the left two-thirds; recent-activity feed of ingested material on the right third. Payment cards are rendered only for permitted roles.

- **Client 360** — identity header band; tabbed body of Projects, Communications, Meetings, Payments and History. Ingested artefacts show the AI summary collapsed, with the full source expandable.
- **Project Workspace** — left column holds scope, milestones and teams with a reassign control; main column is a chronological stream mixing work logs, chat imports and meeting summaries, so a newcomer reads the project as one narrative.
- **Upload Centre** — client and project selectors sit before the file picker, enforcing BR-01 at the interface. A status list shows each file progressing through parsing, transcription, embedding and extraction.
- **AI Assistant** — conversation column with a persistent citation panel; numbered source chips open the original message, transcript segment or work log in a side panel. An advisory notice implements BR-03.

3.3 State Management

State	Approach
Session and identity	Global auth context holding token, identity and resolved permission set; drives conditional rendering.
Server data	Query cache per resource, keyed by entity id, invalidated on mutation.
Upload progress	Polled status per upload id in local state, reconciled against the uploads list.
Chat session	Message array with answers, citations and pending state; not persisted beyond the session.
UI state	Local component state for tabs, filters, modals and expanded summaries; never lifted globally.

3.4 API-to-UI Data Binding

Screen	Endpoint(s)	Bound To
Dashboard	GET /dashboard	Counter cards, project status list, activity feed
Leads	GET, POST, PATCH /leads	Stage columns and lead cards

Screen	Endpoint(s)	Bound To
Client 360	GET /clients/{id}	Header band and five tabs
Project Workspace	GET /projects/{id}, POST /projects/{id}/assignments	Scope panel, team panel, activity stream
Upload Centre	POST /uploads, GET /uploads/{id}/status	Drop zone, tagging selectors, status list
Work Log Entry	POST /work-logs	Entry form and confirmation toast
Payments	GET, POST /payments	Invoice table and totals strip
Search	GET /search	Result list with source-type badges
AI Assistant	POST /chat	Answer bubble, citation chips, source side panel

3.5 Access Control in the Interface

- The permission set resolved at login decides which navigation items render; hidden routes are also blocked server-side, since interface hiding is presentation, not security.
- Payment figures never appear in dashboard counters, search results or chat answers for roles without payment permission.
- Search and chat are available to every role — restricted users simply receive fewer sources rather than an error.

4. Context Handoffs

Bundling the codebase and the three documents above (using a tool such as Repomix) so the AI always has full project context when writing code.

4.1 What Is Bundled

Handoff	Bundle Contents	Produces
Research → Product Profile	Raw research notes, interview findings, competitor observations	Section 1
Product Profile → SRS	Personas, JTBD, business rules, constraints, feature scope	Section 2
SRS → Backend code	Functional requirements, schema, API structure, NFRs	Backend modules
Backend → PX Strategy	SRS plus the built backend repository and live API contract	Section 3
PX Strategy → Frontend code	PX Strategy plus backend repository and component conventions	Frontend modules

4.2 Handoff Rules

- Every item in the outgoing document carries an identifier (BR, CON, FR) that later work can reference precisely.
- Nothing exists downstream that cannot be traced to an identifier upstream; anything untraceable is scope creep and is rejected.
- Open questions are resolved and written down before handoff, not carried forward as assumptions.
- The bundle is regenerated fresh at each handoff so the AI never works from a stale view of the codebase.

4.3 Build Sequence

Week	Deliverable	Bundle Fed to the AI
1	SQL Server schema; FastAPI backend with auth and RBAC; React dashboard shell with CRUD screens.	SRS §2.3, §2.5, §2.6
2	Upload interface and parsers for WhatsApp, Telegram, email and Meet; work-log entry.	SRS FR-ING + backend repo
3	Whisper transcription; chunking and BGE embedding pipeline; Qdrant storage.	SRS FR-AI-01 to 03 + backend repo
4	Summarisation with Llama/Mistral; entity extraction; Neo4j population.	SRS FR-AI-04 to 07 + backend repo
5	Semantic search; RAG chatbot with vector and graph retrieval and citations, RBAC enforced.	SRS FR-RAG + PX §3.4
6	Payment module; dashboard reporting; polish; deploy and self-test.	PX Strategy + full repo

4.4 Risks

Risk	Mitigation
Legal and consent around client data	Manual, deliberate upload only; written internal data-handling policy.
Security of concentrated client history	Strict RBAC, encryption at rest and in transit, access audit logs.
AI hallucination	Source citations on every answer; answer only from retrieved context; output is advisory (BR-03).
Employee adoption resistance	Minimal manual effort; positioned as protecting the team, not surveillance (BR-09).
Limited compute	Small open-weight models, batch and asynchronous processing.
Scope creep in a six-week build	No requirement enters a stage without an upstream identifier (§4.2).